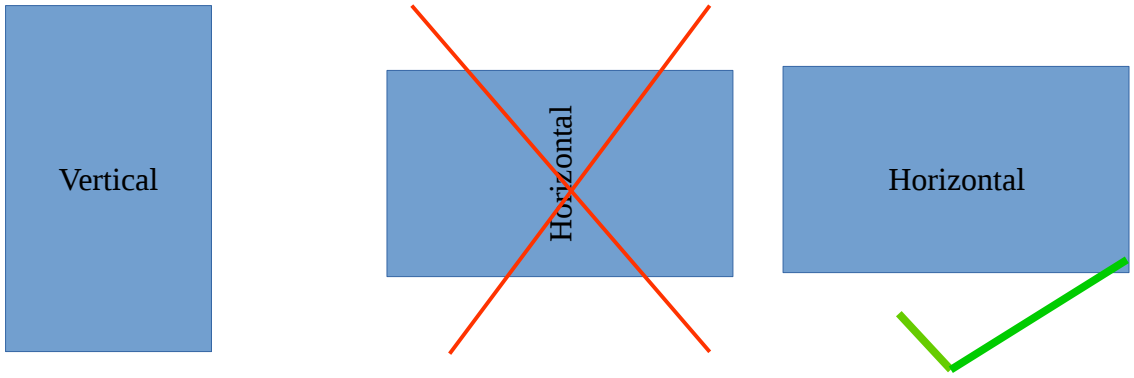
# Introduction to Multiple Screen size and orientation

While making an application we should consider a) where the application will run and b) who is the user those are going to use that.

We are in mobile age, most of the application are being built for mobile. Even the web pages now should have RWD (Responsive Web Design). Google has already created the algorithm to lower the ranking in their search for the webpages which are not in responsive design. In case of android application we should consider many types of screen size while developing it. Because android is available for Phone, Desktop, Tablet, Auto, Wearable and TV. For example if we are creating application for Mobile Phone or Tablet then we should consider the screen resolutions which should be supported. Like as Mobile can have: 768x1024, 800x1280, 648x1280, 360x640 etc... and Table can have: 1280x800, 1024x600, 2048x1536 etc...

Normally, when we talk about Mobile phone, it has more height than width as it is designed vertically. Therefore we may say we can design our application vertically but what if the user flip the screen and make it horizontal. Please see the screen on position 1 and position two.




Our screen should also be responsive to tackle horizontal position of the mobile. This is why android comes out with many layout and vertical/horizontal orientation to tackle those needs.

# User Interface and experiences

User interface is everything that a user can see and interact. Android provides a variety of pre-build UI components such as structured layout objects and UI controls that allow you to build the graphical user interface for your app. Android also provides other UI modules for special interfaces such as dialogs, notifications, and menus.

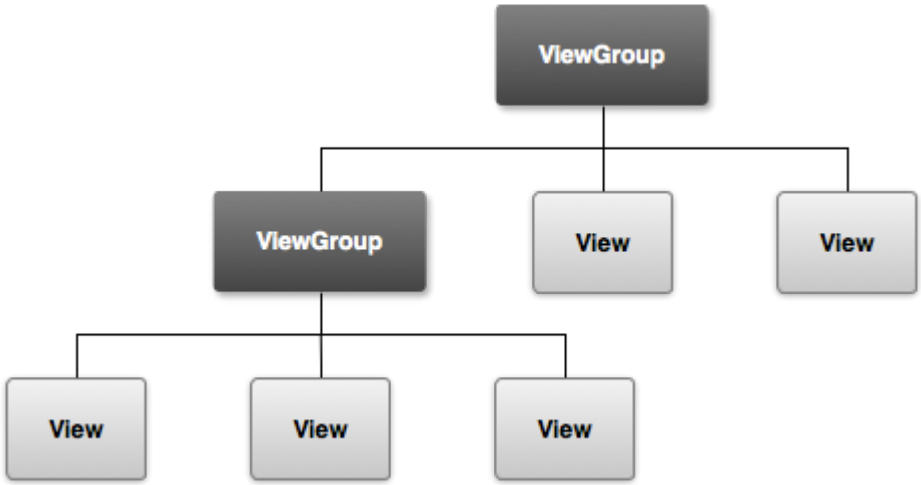
**UI Overview:**

All user interface elements in an Android app are built using View and ViewGroup objects. A View is an object that draws something on the screen that the user can interact with. A ViewGroup is an object that holds other View (and ViewGroup) objects in order to define the layout of the interface.

Android provides a collection of both View and ViewGroup subclasses that offer you common input controls (such as buttons and text fields) and various layout models (such as a linear or relative layout).

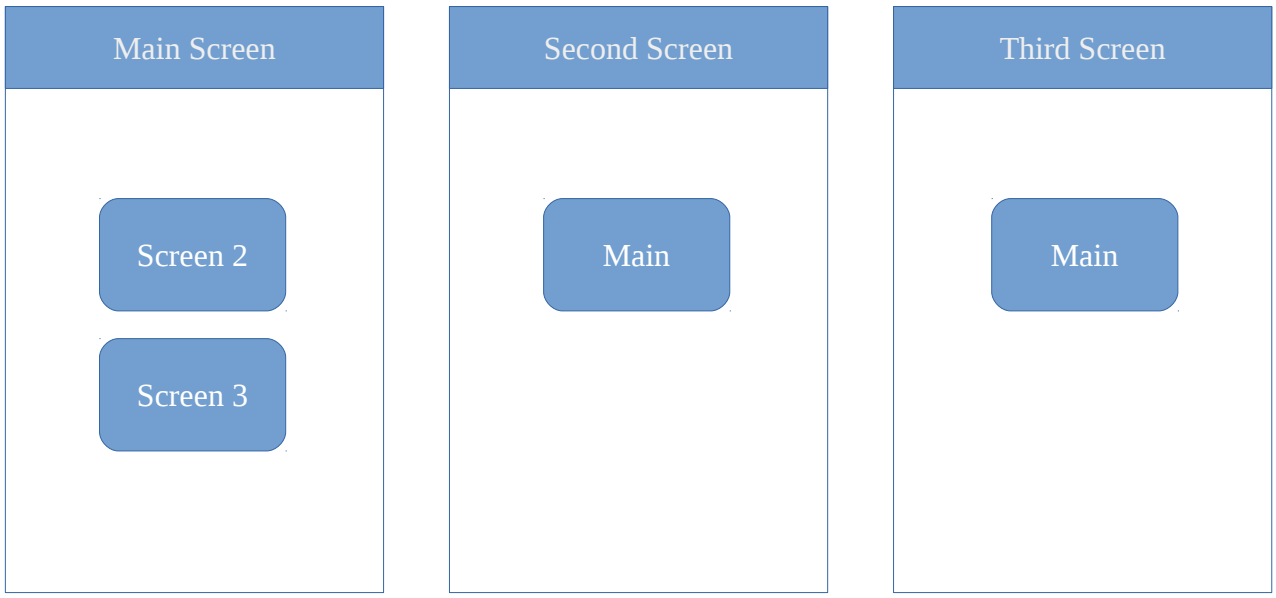
**User Interface Layout**

The user interface for each component of your app is defined using a hierarchy of View and ViewGroupobjects, as shown in figure 1. Each view group is an invisible container that organizes child views, while the child views may be input controls or other widgets that draw some part of the UI. This hierarchy tree can be as simple or complex as you need it to be (but simplicity is best for performance).



**User Interface Scenario:**

For a practical session to understand Layout, Views, Intent, Activity we have used 3 user interface screen. Like Below




Main Activity Screen 2 Screen 3

So, on the process of this practical we created user interface where we have designed

1) Layout for each screen
2) Buttons with event where if screen2 is clicked on main activity it will take us to second screen and so on.

# Android XML Layouts, Resources and Styles

A layout defines the visual structure for a user interface, such as the UI for an activity or app widget. You can declare a layout in two ways:

- **Declare UI elements in XML**. Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts.
- **Instantiate layout elements at runtime**. Your application can create View and ViewGroup objects (and manipulate their properties) programmatically.

The Android framework gives you the flexibility to use either or both of these methods for declaring and managing your application's UI. For example, you could declare your application's default layouts in XML, including the screen elements that will appear in them and their properties. You could then add code in your application that would modify the state of the screen objects, including those declared in XML, at run time.

### Creating Layout for Activity

All the layout file will be inside /res/layout directory.

Just create XML Layout file, once the xml file is created you can drag and drop the components.

Using Android's XML vocabulary, you can quickly design UI layouts and the screen elements they contain, in the same way you create web pages in HTML — with a series of nested elements.

Each layout file must contain exactly one root element, which must be a View or ViewGroup object. Once you've defined the root element, you can add additional layout objects or widgets as child elements to gradually build a View hierarchy that defines your layout. For example, here's an XML layout that uses a vertical LinearLayoutto hold a `TextView` and a Button:

**Sample:**

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
                    android:layout_width="match_parent"
                    android:layout_height="match_parent"
                    android:orientation="vertical" >
    <TextView android:id="@+id/text"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Text view sample text" />
    <Button android:id="@+id/button"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Text Button" />
</LinearLayout>
```

# Layouts

A layout defines the visual structure for a user interface, such as the UI for an activity or app widget. You can declare a layout in two ways:

- **Declare UI elements in XML**. Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts.
- **Instantiate layout elements at runtime**. Your application can create View and ViewGroup objects (and manipulate their properties) programmatically.

The Android framework gives you the flexibility to use either or both of these methods for declaring and managing your application's UI. For example, you could declare your application's default layouts in XML, including the screen elements that will appear in them and their properties. You could then add code in your application that would modify the state of the screen objects, including those declared in XML, at run time.

The advantage to declaring your UI in XML is that it enables you to better separate the presentation of your application from the code that controls its behavior. Your UI descriptions are external to your application code, which means that you can modify or adapt it without having to modify your source code and recompile. For example, you can create XML layouts for different screen orientations, different device screen sizes, and different languages. Additionally, declaring the layout in XML makes it easier to visualize the structure of your UI, so it's easier to debug problems. As such, this document focuses on teaching you how to declare your layout in XML. If you're interested in instantiating View objects at runtime, refer to the ViewGroup and View class references.

- You should also try the Hierarchy Viewer tool, for debugging layouts — it reveals layout property values, draws wireframes with padding/margin indicators, and full rendered views while you debug on the emulator or device.
- The layoutopt tool lets you quickly analyze your layouts and hierarchies for inefficiencies or other problems.

In general, the XML vocabulary for declaring UI elements closely follows the structure and naming of the classes and methods, where element names correspond to class names and attribute names correspond to methods. In fact, the correspondence is often so direct that you can guess what XML attribute corresponds to a class method, or guess what class corresponds to a given XML element. However, note that not all vocabulary is identical. In some cases, there are slight naming differences. For example, the EditText element has a text attribute that corresponds to EditText.setText().

> **Tip:** Learn more about different layout types in Common Layout Objects.

## Write the XML

Using Android's XML vocabulary, you can quickly design UI layouts and the screen elements they contain, in the same way you create web pages in HTML — with a series of nested elements.

Each layout file must contain exactly one root element, which must be a View or ViewGroup object. Once you've defined the root element, you can add additional layout objects or widgets

as child elements to gradually build a View hierarchy that defines your layout. For example, here's an XML layout that uses a vertical LinearLayout to hold a TextView and a Button:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:orientation="vertical" >
    <TextView android:id="@+id/text"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Hello, I am a Button" />
</LinearLayout>
```

After you've declared your layout in XML, save the file with the .xml extension, in your Android project's res/layout/ directory, so it will properly compile.

More information about the syntax for a layout XML file is available in the Layout Resources document.

# Load the XML Resource

When you compile your application, each XML layout file is compiled into a View resource. You should load the layout resource from your application code, in your Activity.onCreate() callback implementation. Do so by calling setContentView(), passing it the reference to your layout resource in the form of: R.layout.*layout_file_name*. For example, if your XML layout is saved as main_layout.xml, you would load it for your Activity like so:

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main_layout);
}
```

The onCreate() callback method in your Activity is called by the Android framework when your Activity is launched (see the discussion about lifecycles, in the Activities document).

# Attributes

Every View and ViewGroup object supports their own variety of XML attributes. Some attributes are specific to a View object (for example, TextView supports the textSize attribute), but these attributes are also inherited by any View objects that may extend this class. Some are common to all View objects, because they are inherited from the root View class (like the id attribute). And, other attributes are considered "layout parameters," which are attributes that describe certain layout orientations of the View object, as defined by that object's parent ViewGroup object.

## ID

Any View object may have an integer ID associated with it, to uniquely identify the View within the tree. When the application is compiled, this ID is referenced as an integer, but the ID is typically assigned in the layout XML file as a string, in the id attribute. This is an XML attribute

common to all View objects (defined by the View class) and you will use it very often. The syntax for an ID, inside an XML tag is:

```
android:id="@+id/my_button"
```

The at-symbol (@) at the beginning of the string indicates that the XML parser should parse and expand the rest of the ID string and identify it as an ID resource. The plus-symbol (+) means that this is a new resource name that must be created and added to our resources (in the R.java file). There are a number of other ID resources that are offered by the Android framework. When referencing an Android resource ID, you do not need the plus-symbol, but must add the android package namespace, like so:

```
android:id="@android:id/empty"
```

With the android package namespace in place, we're now referencing an ID from the android.R resources class, rather than the local resources class.

In order to create views and reference them from the application, a common pattern is to:

1. Define a view/widget in the layout file and assign it a unique ID:

```
<Button android:id="@+id/my_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/my_button_text"/>
```

2. Then create an instance of the view object and capture it from the layout (typically in the onCreate() method):

```
Button myButton = (Button) findViewById(R.id.my_button);
```

Defining IDs for view objects is important when creating a RelativeLayout. In a relative layout, sibling views can define their layout relative to another sibling view, which is referenced by the unique ID.

An ID need not be unique throughout the entire tree, but it should be unique within the part of the tree you are searching (which may often be the entire tree, so it's best to be completely unique when possible).

# Layout Parameters

XML layout attributes named layout_*something* define layout parameters for the View that are appropriate for the ViewGroup in which it resides.

Every ViewGroup class implements a nested class that extends ViewGroup.LayoutParams. This subclass contains property types that define the size and position for each child view, as appropriate for the view group. As you can see in figure 1, the parent view group defines layout parameters for each child view (including the child view group).

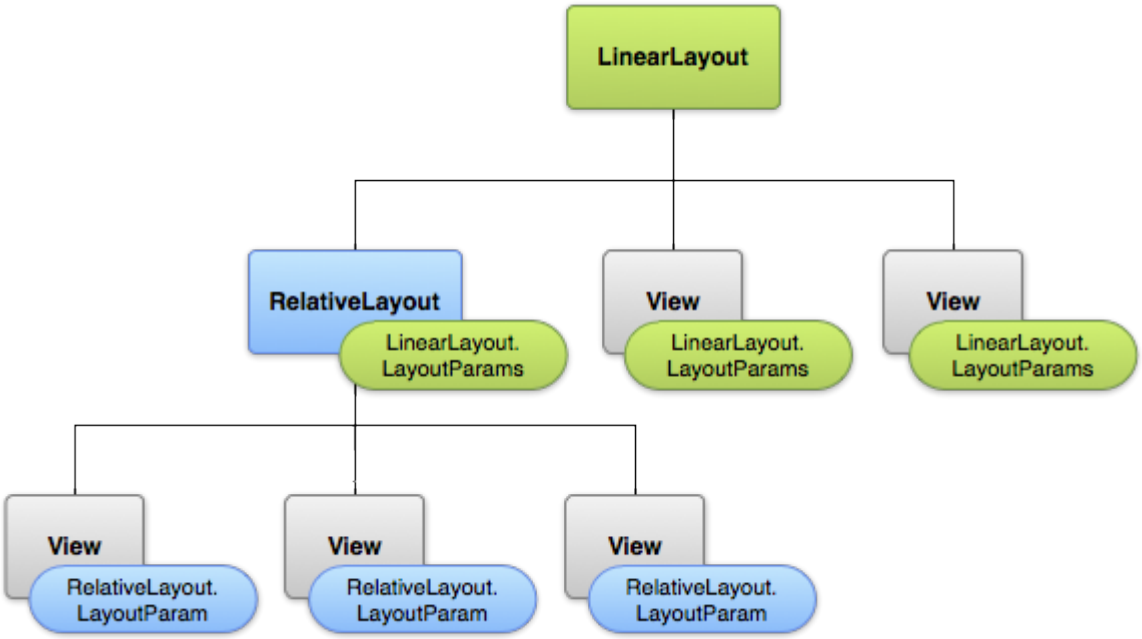



**Figure 1.** Visualization of a view hierarchy with layout parameters associated with each view.

Note that every LayoutParams subclass has its own syntax for setting values. Each child element must define LayoutParams that are appropriate for its parent, though it may also define different LayoutParams for its own children.

All view groups include a width and height (layout_width and layout_height), and each view is required to define them. Many LayoutParams also include optional margins and borders.

You can specify width and height with exact measurements, though you probably won't want to do this often. More often, you will use one of these constants to set the width or height:

- **wrap_content** tells your view to size itself to the dimensions required by its content.
- **match_parent** tells your view to become as big as its parent view group will allow.

In general, specifying a layout width and height using absolute units such as pixels is not recommended. Instead, using relative measurements such as density-independent pixel units (**dp**), **wrap_content**, or **match_parent**, is a better approach, because it helps ensure that your application will display properly across a variety of device screen sizes. The accepted measurement types are defined in the Available Resources document.

# Layout Position

The geometry of a view is that of a rectangle. A view has a location, expressed as a pair of *left* and *top* coordinates, and two dimensions, expressed as a width and a height. The unit for location and dimensions is the pixel.

It is possible to retrieve the location of a view by invoking the methods getLeft() and getTop(). The former returns the left, or X, coordinate of the rectangle representing the view. The latter returns the top, or Y, coordinate of the rectangle representing the view. These methods both return the location of the view relative to its parent. For instance, when getLeft() returns 20, that means the view is located 20 pixels to the right of the left edge of its direct parent.

In addition, several convenience methods are offered to avoid unnecessary computations, namely getRight() and getBottom(). These methods return the coordinates of the right and bottom edges of the rectangle representing the view. For instance, calling getRight() is similar to the following computation: getLeft() + getWidth().

# Size, Padding and Margins

The size of a view is expressed with a width and a height. A view actually possess two pairs of width and height values.

The first pair is known as *measured width* and *measured height*. These dimensions define how big a view wants to be within its parent. The measured dimensions can be obtained by calling getMeasuredWidth() and getMeasuredHeight().

The second pair is simply known as *width* and *height*, or sometimes *drawing width* and *drawing height*. These dimensions define the actual size of the view on screen, at drawing time and after layout. These values may, but do not have to, be different from the measured width and height. The width and height can be obtained by calling getWidth() and getHeight().

To measure its dimensions, a view takes into account its padding. The padding is expressed in pixels for the left, top, right and bottom parts of the view. Padding can be used to offset the content of the view by a specific number of pixels. For instance, a left padding of 2 will push the view's content by 2 pixels to the right of the left edge. Padding can be set using the setPadding(int, int, int, int) method and queried by callinggetPaddingLeft(), getPaddingTop(), getPaddingRight() and getPaddingBottom().

Even though a view can define a padding, it does not provide any support for margins. However, view groups provide such a support. Refer toViewGroup and ViewGroup.MarginLayoutParams for further information.

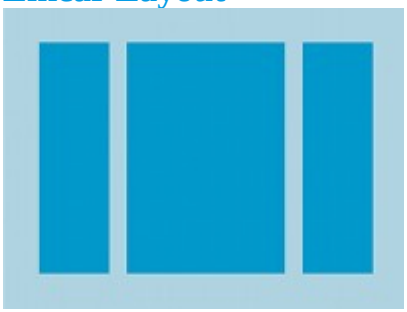
For more information about dimensions, see Dimension Values.

# Common Layouts

Each subclass of the ViewGroup class provides a unique way to display the views you nest within it. Below are some of the more common layout types that are built into the Android platform.

> **Note:** Although you can nest one or more layouts within another layout to acheive your UI design, you should strive to keep your layout hierarchy as shallow as possible. Your layout draws faster if it has fewer nested layouts (a wide view hierarchy is better than a deep view hierarchy).

### Linear Layout



A layout that organizes its children into a single horizontal or vertical row. It creates a scrollbar if the length of the window exceeds the length of the screen.

### Relative Layout

Enables you to specify the location of child objects relative to each other (child A to the left of child B) or to the parent (aligned to the top of the parent).

### Web View


Displays web pages.

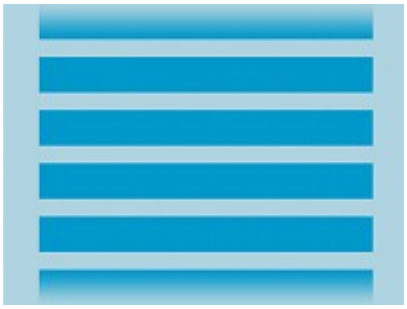
# Building Layouts with an Adapter

---

When the content for your layout is dynamic or not pre-determined, you can use a layout that subclasses AdapterView to populate the layout with views at runtime. A subclass of the AdapterView class uses an Adapter to bind data to its layout. The Adapter behaves as a middleman between the data source and the AdapterView layout—the Adapter retrieves the data (from a source such as an array or a database query) and converts each entry into a view that can be added into the AdapterView layout.

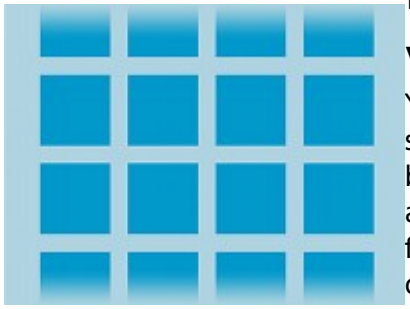
Common layouts backed by an adapter include:

### List View



Displays a scrolling single column list.

### Grid View



Displays a scrolling grid of columns and rows.

## Filling an adapter view with data

You can populate an AdapterView such as ListView or GridView by binding the AdapterView instance to an Adapter, which retrieves data from an external source and creates a View that represents each data entry.

Android provides several subclasses of Adapter that are useful for retrieving different kinds of data and building views for an AdapterView. The two most common adapters are:

ArrayAdapter

Use this adapter when your data source is an array. By default, ArrayAdapter creates a view for each array item by calling toString() on each item and placing the contents in a TextView.

For example, if you have an array of strings you want to display in a ListView, initialize a new ArrayAdapter using a constructor to specify the layout for each string and the string array:

```
ArrayAdapter<String> adapter = new ArrayAdapter<String>(this,

        android.R.layout.simple_list_item_1, myStringArray);
```

The arguments for this constructor are:

- Your app Context
- The layout that contains a TextView for each string in the array
- The string array

Then simply call setAdapter() on your ListView:

```
ListView listView = (ListView) findViewById(R.id.listview);

listView.setAdapter(adapter);
```

To customize the appearance of each item you can override the toString() method for the objects in your array. Or, to create a view for each item that's something other than a TextView (for example, if you want an ImageView for each array item), extend the ArrayAdapter class and override getView() to return the type of view you want for each item.

## SimpleCursorAdapter

Use this adapter when your data comes from a Cursor. When using SimpleCursorAdapter, you must specify a layout to use for each row in the Cursor and which columns in the Cursor should be inserted into which views of the layout. For example, if you want to create a list of people's names and phone numbers, you can perform a query that returns a Cursor containing a row for each person and columns for the names and numbers. You then create a string array specifying which columns from the Cursor you want in the layout for each result and an integer array specifying the corresponding views that each column should be placed:

```
String[] fromColumns = {ContactsContract.Data.DISPLAY_NAME,

                        ContactsContract.CommonDataKinds.Phone.NUMBER};

int[] toViews = {R.id.display_name, R.id.phone_number};
```

When you instantiate the SimpleCursorAdapter, pass the layout to use for each result, the Cursor containing the results, and these two arrays:

```
SimpleCursorAdapter adapter = new SimpleCursorAdapter(this,

        R.layout.person_name_and_number, cursor, fromColumns, toViews, 0);
```

```
ListView listView = getListView();

listView.setAdapter(adapter);
```

The SimpleCursorAdapter then creates a view for each row in the Cursor using the provided layout by inserting each fromColumns item into the corresponding toViews view.

.

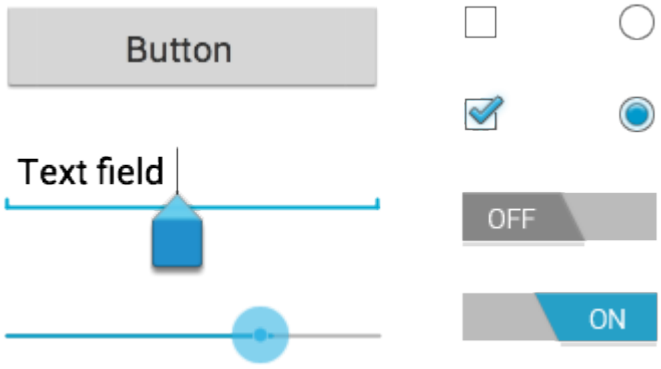
If, during the course of your application's life, you change the underlying data that is read by your adapter, you should call notifyDataSetChanged(). This will notify the attached view that the data has been changed and it should refresh itself.

## Handling click events

You can respond to click events on each item in an AdapterView by implementing the AdapterView.OnItemClickListener interface. For example:

```
// Create a message handling object as an anonymous class.
private OnItemClickListener mMessageClickedHandler = new OnItemClickListener() {
    public void onItemClick(AdapterView parent, View v, int position, long id) {
        // Do something in response to the click
    }
};
listView.setOnItemClickListener(mMessageClickedHandler);
```

# Input Controls




Input controls are the interactive components in your app's user interface. Android provides a wide variety of controls you can use in your UI, such as buttons, text fields, seek bars, checkboxes, zoom buttons, toggle buttons, and many more.

Adding an input control to your UI is as simple as adding an XML element to your XML layout. For example, here's a layout with a text field and button:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="horizontal">
    <EditText android:id="@+id/edit_message"
```

```
        android:layout_weight="1"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="@string/edit_message" />
    <Button android:id="@+id/button_send"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/button_send"
        android:onClick="sendMessage" />
</LinearLayout>
```

Each input control supports a specific set of input events so you can handle events such as when the user enters text or touches a button.

# Common Controls

Here's a list of some common controls that you can use in your app. Follow the links to learn more about using each one.

> **Note:** Android provides several more controls than are listed here. Browse the android.widget package to discover more. If your app requires a specific kind of input control, you can build your own custom components.

| Control Type | Description | Related Classes |
|---|---|---|
| Button | A push-button that can be pressed, or clicked, by the user to perform an action. | Button |
| Text field | An editable text field. You can use the AutoCompleteTextView widget to create a text entry widget that provides auto-complete suggestions | EditText, AutoCompleteTextView |
| Checkbox | An on/off switch that can be toggled by the user. You should use checkboxes when presenting users with a group of selectable options that are not mutually exclusive. | CheckBox |
| Radio button | Similar to checkboxes, except that only one option can be selected in the group. | RadioGroup RadioButton |
| Toggle button | An on/off button with a light indicator. | ToggleButton |
| Spinner | A drop-down list that allows users to select one value from a set. | Spinner |
| Pickers | A dialog for users to select a single value for a set by using up/down buttons or via a swipe gesture. Use a DatePickercode> widget to enter the values for the date (month, day, year) or a TimePickerwidget to enter the values for a time (hour, minute, AM/PM), which will be formatted automatically for the user's locale. | |

# Input Events

On Android, there's more than one way to intercept the events from a user's interaction with your application. When considering events within your user interface, the approach is to capture the events from the specific View object that the user interacts with. The View class provides the means to do so.

Within the various View classes that you'll use to compose your layout, you may notice several public callback methods that look useful for UI events. These methods are called by the Android framework when the respective action occurs on that object. For instance, when a View (such as a Button) is touched, the onTouchEvent() method is called on that object. However, in order to intercept this, you must extend the class and override the method. However, extending every View object in order to handle such an event would not be practical. This is why the View class also contains a collection of nested interfaces with callbacks that you can much more easily define. These interfaces, called event listeners, are your ticket to capturing the user interaction with your UI.

While you will more commonly use the event listeners to listen for user interaction, there may come a time when you do want to extend a View class, in order to build a custom component. Perhaps you want to extend the Button class to make something more fancy. In this case, you'll be able to define the default event behaviors for your class using the class event handlers.

## Event Listeners

An event listener is an interface in the View class that contains a single callback method. These methods will be called by the Android framework when the View to which the listener has been registered is triggered by user interaction with the item in the UI.

Included in the event listener interfaces are the following callback methods:

onClick()

> From View.OnClickListener. This is called when the user either touches the item (when in touch mode), or focuses upon the item with the navigation-keys or trackball and presses the suitable "enter" key or presses down on the trackball.

onLongClick()

> From View.OnLongClickListener. This is called when the user either touches and holds the item (when in touch mode), or focuses upon the item with the navigation-keys or trackball and presses and holds the suitable "enter" key or presses and holds down on the trackball (for one second).

onFocusChange()

> From View.OnFocusChangeListener. This is called when the user navigates onto or away from the item, using the navigation-keys or trackball.

onKey()

From View.OnKeyListener. This is called when the user is focused on the item and presses or releases a hardware key on the device.

onTouch()

From View.OnTouchListener. This is called when the user performs an action qualified as a touch event, including a press, a release, or any movement gesture on the screen (within the bounds of the item).

onCreateContextMenu()

From View.OnCreateContextMenuListener. This is called when a Context Menu is being built (as the result of a sustained "long click"). See the discussion on context menus in the Menus developer guide.

These methods are the sole inhabitants of their respective interface. To define one of these methods and handle your events, implement the nested interface in your Activity or define it as an anonymous class. Then, pass an instance of your implementation to the respective View.set...Listener()method. (E.g., call setOnClickListener() and pass it your implementation of the OnClickListener.)

The example below shows how to register an on-click listener for a Button.

```
// Create an anonymous implementation of OnClickListener
private OnClickListener mCorkyListener = new OnClickListener() {
    public void onClick(View v) {
      // do something when the button is clicked
    }
};
protected void onCreate(Bundle savedValues) {
    ...
    // Capture our button from layout
    Button button = (Button)findViewById(R.id.corky);
    // Register the onClick listener with the implementation above
    button.setOnClickListener(mCorkyListener);
    ...
}
```

You may also find it more convenient to implement OnClickListener as a part of your Activity. This will avoid the extra class load and object allocation. For example:

```
public class ExampleActivity extends Activity implements OnClickListener {
    protected void onCreate(Bundle savedValues) {
        ...
        Button button = (Button)findViewById(R.id.corky);
        button.setOnClickListener(this);
    }
    // Implement the OnClickListener callback
    public void onClick(View v) {
      // do something when the button is clicked
    }
    ...
}
```

Notice that the onClick() callback in the above example has no return value, but some other event listener methods must return a boolean. The reason depends on the event. For the few that do, here's why:

- onLongClick() - This returns a boolean to indicate whether you have consumed the event and it should not be carried further. That is, return *true* to indicate that you have handled the event and it should stop here; return *false* if you have not handled it and/or the event should continue to any other on-click listeners.
- onKey() - This returns a boolean to indicate whether you have consumed the event and it should not be carried further. That is, return *true* to indicate that you have handled the event and it should stop here; return *false* if you have not handled it and/or the event should continue to any other on-key listeners.
- onTouch() - This returns a boolean to indicate whether your listener consumes this event. The important thing is that this event can have multiple actions that follow each other. So, if you return *false* when the down action event is received, you indicate that you have not consumed the event and are also not interested in subsequent actions from this event. Thus, you will not be called for any other actions within the event, such as a finger gesture, or the eventual up action event.

Remember that hardware key events are always delivered to the View currently in focus. They are dispatched starting from the top of the View hierarchy, and then down, until they reach the appropriate destination. If your View (or a child of your View) currently has focus, then you can see the event travel through the dispatchKeyEvent() method. As an alternative to capturing key events through your View, you can also receive all of the events inside your Activity with onKeyDown() and onKeyUp().

Also, when thinking about text input for your application, remember that many devices only have software input methods. Such methods are not required to be key-based; some may use voice input, handwriting, and so on. Even if an input method presents a keyboard-like interface, it will generally **not** trigger the onKeyDown() family of events. You should never build a UI that requires specific key presses to be controlled unless you want to limit your application to devices with a hardware keyboard. In particular, do not rely on these methods to validate input when the user presses the return key; instead, use actions like IME_ACTION_DONE to signal the input method how your application expects to react, so it may change its UI in a meaningful way. Avoid assumptions about how a software input method should work and just trust it to supply already formatted text to your application.

> **Note:** Android will call event handlers first and then the appropriate default handlers from the class definition second. As such, returning *true* from these event listeners will stop the propagation of the event to other event listeners and will also block the callback to the default event handler in the View. So be certain that you want to terminate the event when you return *true*.

# Event Handlers

If you're building a custom component from View, then you'll be able to define several callback methods used as default event handlers. In the document about Custom Components, you'll learn see some of the common callbacks used for event handling, including:

- onKeyDown(int, KeyEvent) - Called when a new key event occurs.

- onKeyUp(int, KeyEvent) - Called when a key up event occurs.
- onTrackballEvent(MotionEvent) - Called when a trackball motion event occurs.
- onTouchEvent(MotionEvent) - Called when a touch screen motion event occurs.
- onFocusChanged(boolean, int, Rect) - Called when the view gains or loses focus.

There are some other methods that you should be aware of, which are not part of the View class, but can directly impact the way you're able to handle events. So, when managing more complex events inside a layout, consider these other methods:

- Activity.dispatchTouchEvent(MotionEvent) - This allows your Activity to intercept all touch events before they are dispatched to the window.
- ViewGroup.onInterceptTouchEvent(MotionEvent) - This allows a ViewGroup to watch events as they are dispatched to child Views.
- ViewParent.requestDisallowInterceptTouchEvent(boolean) - Call this upon a parent View to indicate that it should not intercept touch events with onInterceptTouchEvent(MotionEvent).

# Touch Mode

When a user is navigating a user interface with directional keys or a trackball, it is necessary to give focus to actionable items (like buttons) so the user can see what will accept input. If the device has touch capabilities, however, and the user begins interacting with the interface by touching it, then it is no longer necessary to highlight items, or give focus to a particular View. Thus, there is a mode for interaction named "touch mode."

For a touch-capable device, once the user touches the screen, the device will enter touch mode. From this point onward, only Views for whichisFocusableInTouchMode() is true will be focusable, such as text editing widgets. Other Views that are touchable, like buttons, will not take focus when touched; they will simply fire their on-click listeners when pressed.

Any time a user hits a directional key or scrolls with a trackball, the device will exit touch mode, and find a view to take focus. Now, the user may resume interacting with the user interface without touching the screen.

The touch mode state is maintained throughout the entire system (all windows and activities). To query the current state, you can callisInTouchMode() to see whether the device is currently in touch mode.

# Handling Focus

The framework will handle routine focus movement in response to user input. This includes changing the focus as Views are removed or hidden, or as new Views become available. Views indicate their willingness to take focus through the isFocusable() method. To change whether a View can take focus, call setFocusable(). When in touch mode, you may query whether a View allows focus with isFocusableInTouchMode(). You can change this with setFocusableInTouchMode().

Focus movement is based on an algorithm which finds the nearest neighbor in a given direction. In rare cases, the default algorithm may not match the intended behavior of the developer. In these situations, you can provide explicit overrides with the following XML attributes in the layout file:**nextFocusDown**, **nextFocusLeft**, **nextFocusRight**, and

**nextFocusUp**. Add one of these attributes to the View *from* which the focus is leaving. Define the value of the attribute to be the id of the View *to* which focus should be given. For example:

```
<LinearLayout
      android:orientation="vertical"
      ... >
   <Button android:id="@+id/top"
              android:nextFocusUp="@+id/bottom"
              ... />
   <Button android:id="@+id/bottom"
              android:nextFocusDown="@+id/top"
              ... />
</LinearLayout>
```

Ordinarily, in this vertical layout, navigating up from the first Button would not go anywhere, nor would navigating down from the second Button. Now that the top Button has defined the bottom one as the **nextFocusUp** (and vice versa), the navigation focus will cycle from top-to-bottom and bottom-to-top.

If you'd like to declare a View as focusable in your UI (when it is traditionally not), add the android:focusable XML attribute to the View, in your layout declaration. Set the value **true**. You can also declare a View as focusable while in Touch Mode with android:focusableInTouchMode.

To request a particular View to take focus, call requestFocus().

To listen for focus events (be notified when a View receives or looses focus), use onFocusChange(), as discussed in the Event Listeners section, above.

# Menus

Menus are a common user interface component in many types of applications. To provide a familiar and consistent user experience, you should use the Menu APIs to present user actions and other options in your activities.

Beginning with Android 3.0 (API level 11), Android-powered devices are no longer required to provide a dedicated *Menu* button. With this change, Android apps should migrate away from a dependence on the traditional 6-item menu panel and instead provide an app bar to present common user actions.

Although the design and user experience for some menu items have changed, the semantics to define a set of actions and options is still based on the Menu APIs. This guide shows how to create the three fundamental types of menus or action presentations on all versions of Android:

### Options menu and app bar

> The options menu is the primary collection of menu items for an activity. It's where you should place actions that have a global impact on the app, such as "Search," "Compose email," and "Settings."

See the section about Creating an Options Menu.

### Context menu and contextual action mode

A context menu is a floating menu that appears when the user performs a long-click on an element. It provides actions that affect the selected content or context frame.

The contextual action mode displays action items that affect the selected content in a bar at the top of the screen and allows the user to select multiple items.

See the section about Creating Contextual Menus.

### Popup menu

A popup menu displays a list of items in a vertical list that's anchored to the view that invoked the menu. It's good for providing an overflow of actions that relate to specific content or to provide options for a second part of a command. Actions in a popup menu should **not** directly affect the corresponding content—that's what contextual actions are for. Rather, the popup menu is for extended actions that relate to regions of content in your activity.

See the section about Creating a Popup Menu.

# Defining a Menu in XML

For all menu types, Android provides a standard XML format to define menu items. Instead of building a menu in your activity's code, you should define a menu and all its items in an XML menu resource. You can then inflate the menu resource (load it as a Menu object) in your activity or fragment.

Using a menu resource is a good practice for a few reasons:

- It's easier to visualize the menu structure in XML.
- It separates the content for the menu from your application's behavioral code.
- It allows you to create alternative menu configurations for different platform versions, screen sizes, and other configurations by leveraging the app resources framework.

To define the menu, create an XML file inside your project's res/menu/ directory and build the menu with the following elements:

<menu>

Defines a Menu, which is a container for menu items. A <menu> element must be the root node for the file and can hold one or more <item> and <group> elements.

<item>

Creates a MenuItem, which represents a single item in a menu. This element may contain a nested <menu> element in order to create a submenu.

<group>

An optional, invisible container for <item> elements. It allows you to categorize menu items so they share properties such as active state and visibility. For more information, see the section about Creating Menu Groups.

Here's an example menu named game_menu.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/new_game"
          android:icon="@drawable/ic_new_game"
          android:title="@string/new_game"
          android:showAsAction="ifRoom"/>
    <item android:id="@+id/help"
          android:icon="@drawable/ic_help"
          android:title="@string/help" />
</menu>
```

The <item> element supports several attributes you can use to define an item's appearance and behavior. The items in the above menu include the following attributes:

android:id

A resource ID that's unique to the item, which allows the application to recognize the item when the user selects it.

android:icon

A reference to a drawable to use as the item's icon.

android:title

A reference to a string to use as the item's title.

android:showAsAction

Specifies when and how this item should appear as an action item in the app bar.

These are the most important attributes you should use, but there are many more available. For information about all the supported attributes, see the Menu Resource document.

You can add a submenu to an item in any menu (except a submenu) by adding a <menu> element as the child of an <item>. Submenus are useful when your application has a lot of functions that can be organized into topics, like items in a PC application's menu bar (File, Edit, View, etc.). For example:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/file"
          android:title="@string/file" >
        <!-- "file" submenu -->
        <menu>
            <item android:id="@+id/create_new"
                  android:title="@string/create_new" />
```

```
            <item android:id="@+id/open"
                  android:title="@string/open" />
        </menu>
    </item>
</menu>
```

To use the menu in your activity, you need to inflate the menu resource (convert the XML resource into a programmable object) using MenuInflater.inflate(). In the following sections, you'll see how to inflate a menu for each menu type.

# Creating an Options Menu

The options menu is where you should include actions and other options that are relevant to the current activity context, such as "Search," "Compose email," and "Settings."

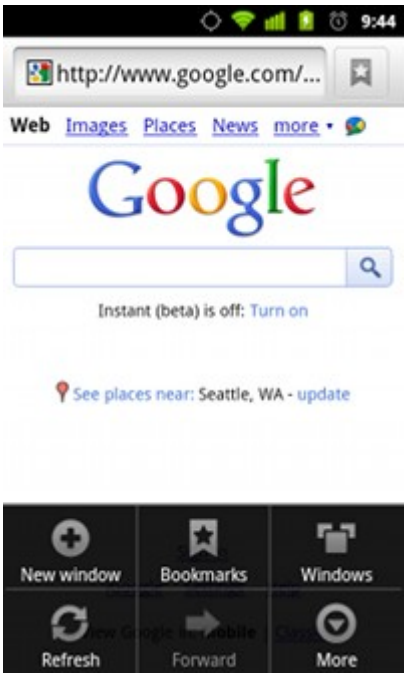



**Figure 1.** Options menu in the Browser, on Android 2.3.

Where the items in your options menu appear on the screen depends on the version for which you've developed your application:

- If you've developed your application for **Android 2.3.x (API level 10) or lower**, the contents of your options menu appear at the bottom of the screen when the user presses the *Menu* button, as shown in figure 1. When opened, the first visible portion is the icon menu, which holds up to six menu items. If your menu includes more than six items, Android places the sixth item and the rest into the overflow menu, which the user can open by selecting*More*.

- If you've developed your application for **Android 3.0 (API level 11) and higher**, items from the options menu are available in the app bar. By default, the system places all items in the action overflow, which the user can reveal with the action overflow icon on the right side of the app bar (or by pressing the device *Menu* button, if available). To enable quick access to important actions, you can promote a few items to appear in the app bar by addingandroid:showAsAction="ifRoom" to the corresponding <item> elements (see figure 2).

  For more information about action items and other app bar behaviors, see the Adding the App Bar training class.

**Figure 2.** The Google Play Movies app, showing a search button and a link to the Google Play Store (plus the action overflow button).

You can declare items for the options menu from either your Activity subclass or a Fragment subclass. If both your activity and fragment(s) declare items for the options menu, they are combined in the UI. The activity's items appear first, followed by those of each fragment in the order in which each fragment is added to the activity. If necessary, you can re-order the menu items with the android:orderInCategory attribute in each <item> you need to move.

To specify the options menu for an activity, override onCreateOptionsMenu() (fragments provide their own onCreateOptionsMenu() callback). In this method, you can inflate your menu resource (defined in XML) into the Menu provided in the callback. For example:

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.game_menu, menu);
    return true;
}
```

You can also add menu items using add() and retrieve items with findItem() to revise their properties with MenuItem APIs.

If you've developed your application for Android 2.3.x and lower, the system calls onCreateOptionsMenu() to create the options menu when the user opens the menu for the first time. If you've developed for Android 3.0 and higher, the system calls onCreateOptionsMenu() when starting the activity, in order to show items to the app bar.

# Handling click events

When the user selects an item from the options menu (including action items in the app bar), the system calls your activity's onOptionsItemSelected() method. This method passes the MenuItem selected. You can identify the item by calling getItemId(), which returns the unique ID for the menu item (defined by the android:id attribute in the menu resource or with an integer given to the add() method). You can match this ID against known menu items to perform the appropriate action. For example:

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
    // Handle item selection
    switch (item.getItemId()) {
        case R.id.new_game:
            newGame();
            return true;
        case R.id.help:
            showHelp();
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
```

When you successfully handle a menu item, return true. If you don't handle the menu item, you should call the superclass implementation of onOptionsItemSelected() (the default implementation returns false).

If your activity includes fragments, the system first calls onOptionsItemSelected() for the activity then for each fragment (in the order each fragment was added) until one returns true or all fragments have been called.

> **Tip:** Android 3.0 adds the ability for you to define the on-click behavior for a menu item in XML, using the android:onClick attribute. The value for the attribute must be the name of a method defined by the activity using the menu. The method must be public and accept a single MenuItemparameter—when the system calls this method, it passes the menu item selected. For more information and an example, see the Menu Resourcedocument.
>
> **Tip:** If your application contains multiple activities and some of them provide the same options menu, consider creating an activity that implements nothing except the onCreateOptionsMenu() and onOptionsItemSelected() methods. Then extend this class for each activity that should share the same options menu. This way, you can manage one set of code for handling menu actions and each descendant class inherits the menu behaviors. If you want to add menu items to one of the descendant activities, override onCreateOptionsMenu() in that activity. Call super.onCreateOptionsMenu(menu) so the original menu items are created, then add new menu items with menu.add(). You can also override the super class's behavior for individual menu items.

## Changing menu items at runtime

After the system calls onCreateOptionsMenu(), it retains an instance of the Menu you populate and will not call onCreateOptionsMenu() again unless the menu is invalidated for some reason. However, you should use onCreateOptionsMenu() only to create the initial menu state and not to make changes during the activity lifecycle.

If you want to modify the options menu based on events that occur during the activity lifecycle, you can do so in the onPrepareOptionsMenu()method. This method passes you the Menu object as it currently exists so you can modify it, such as add, remove, or disable items. (Fragments also provide an onPrepareOptionsMenu() callback.)

On Android 2.3.x and lower, the system calls onPrepareOptionsMenu() each time the user opens the options menu (presses the *Menu* button).

On Android 3.0 and higher, the options menu is considered to always be open when menu items are presented in the app bar. When an event occurs and you want to perform a menu update, you must call invalidateOptionsMenu() to request that the system call onPrepareOptionsMenu().

> **Note:** You should never change items in the options menu based on the View currently in focus. When in touch mode (when the user is not using a trackball or d-pad), views cannot take focus, so you should never use focus as the basis for modifying items in the options menu. If you want to provide menu items that are context-sensitive to a View, use a Context Menu.

# Creating Contextual Menus

A contextual menu offers actions that affect a specific item or context frame in the UI. You can provide a context menu for any view, but they are most often used for items in a ListView, GridView, or other view collections in which the user can perform direct actions on each item.

There are two ways to provide contextual actions:

- In a floating context menu. A menu appears as a floating list of menu items (similar to a dialog) when the user performs a long-click (press and hold) on a view that declares support for a context menu. Users can perform a contextual action on one item at a time.

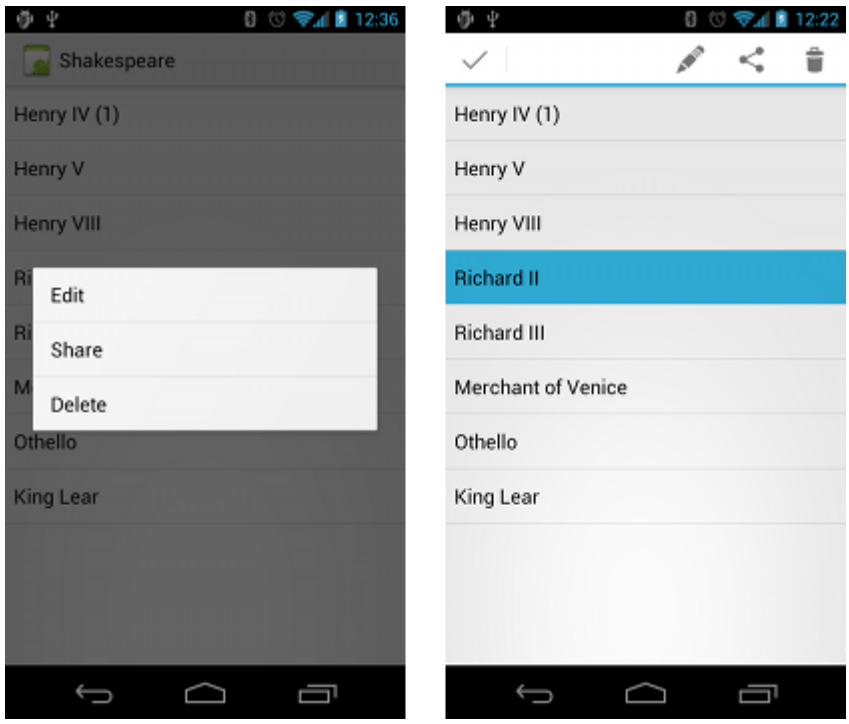



**Figure 3.** Screenshots of a floating context menu (left) and the contextual action bar (right).

- In the contextual action mode. This mode is a system implementation ofActionMode that displays a *contextual action bar* at the top of the screen with action items that affect the selected item(s). When this mode is active, users can perform an action on multiple items at once (if your app allows it).

> **Note:** The contextual action mode is available on Android 3.0 (API level 11) and higher and is the preferred technique for displaying contextual actions when available. If your app supports versions lower than 3.0 then you should fall back to a floating context menu on those devices.

# Creating a floating context menu

To provide a floating context menu:

1. Register the View to which the context menu should be associated by calling registerForContextMenu() and pass it the View.

   If your activity uses a ListView or GridView and you want each item to provide the same context menu, register all items for a context menu by passing the ListView or GridView to registerForContextMenu().
2. Implement the onCreateContextMenu() method in your Activity or Fragment.

   When the registered view receives a long-click event, the system calls your onCreateContextMenu() method. This is where you define the menu items, usually by inflating a menu resource. For example:

```
@Override

public void onCreateContextMenu(ContextMenu menu, View v,

                                ContextMenuInfo menuInfo) {
```

```
    super.onCreateContextMenu(menu, v, menuInfo);

    MenuInflater inflater = getMenuInflater();

    inflater.inflate(R.menu.context_menu, menu);

}
```

MenuInflater allows you to inflate the context menu from a menu resource. The callback method parameters include the View that the user selected and a ContextMenu.ContextMenuInfo object that provides additional information about the item selected. If your activity has several views that each provide a different context menu, you might use these parameters to determine which context menu to inflate.

3. Implement onContextItemSelected().

   When the user selects a menu item, the system calls this method so you can perform the appropriate action. For example:

```
@Override

public boolean onContextItemSelected(MenuItem item) {

    AdapterContextMenuInfo info = (AdapterContextMenuInfo) item.getMenuInfo();

    switch (item.getItemId()) {

        case R.id.edit:

            editNote(info.id);

            return true;

        case R.id.delete:

            deleteNote(info.id);

            return true;

        default:

            return super.onContextItemSelected(item);

    }

}
```

   The getItemId() method queries the ID for the selected menu item, which you should assign to each menu item in XML using the android:idattribute, as shown in the section about Defining a Menu in XML.

   When you successfully handle a menu item, return true. If you don't handle the menu item, you should pass the menu item to the superclass implementation. If your activity includes fragments, the activity receives this callback first. By calling the superclass when unhandled, the system passes the event to the respective callback method in each fragment, one at a time (in the order each fragment was added) until true or false is returned. (The default implementation for Activity and android.app.Fragment return false, so you should always call the superclass when unhandled.)

# Using the contextual action mode

The contextual action mode is a system implementation of ActionMode that focuses user interaction toward performing contextual actions. When a user enables this mode by selecting an item, a *contextual action bar* appears at the top of the screen to present actions the user can perform on the currently selected item(s). While this mode is enabled, the user can select multiple items (if you allow it), deselect items, and continue to navigate within the activity (as much as you're willing to allow). The action mode is disabled and the contextual action bar disappears when the user deselects all items, presses the BACK button, or selects the *Done* action on the left side of the bar.

> **Note:** The contextual action bar is not necessarily associated with the app bar. They operate independently, even though the contextual action bar visually overtakes the app bar position.

For views that provide contextual actions, you should usually invoke the contextual action mode upon one of two events (or both):

- The user performs a long-click on the view.
- The user selects a checkbox or similar UI component within the view.

How your application invokes the contextual action mode and defines the behavior for each action depends on your design. There are basically two designs:

- For contextual actions on individual, arbitrary views.
- For batch contextual actions on groups of items in a ListView or GridView (allowing the user to select multiple items and perform an action on them all).

The following sections describe the setup required for each scenario.

## Enabling the contextual action mode for individual views

If you want to invoke the contextual action mode only when the user selects specific views, you should:

1. Implement the ActionMode.Callback interface. In its callback methods, you can specify the actions for the contextual action bar, respond to click events on action items, and handle other lifecycle events for the action mode.
2. Call startActionMode() when you want to show the bar (such as when the user long-clicks the view).

For example:

1. Implement the ActionMode.Callback interface:

```
private ActionMode.Callback mActionModeCallback = new ActionMode.Callback() {

    // Called when the action mode is created; startActionMode() was called

    @Override

    public boolean onCreateActionMode(ActionMode mode, Menu menu) {

        // Inflate a menu resource providing context menu items
```

```
        MenuInflater inflater = mode.getMenuInflater();
        inflater.inflate(R.menu.context_menu, menu);
        return true;
    }
    // Called each time the action mode is shown. Always called after onCreateActionMode, but
    // may be called multiple times if the mode is invalidated.
    @Override
    public boolean onPrepareActionMode(ActionMode mode, Menu menu) {
        return false; // Return false if nothing is done
    }
    // Called when the user selects a contextual menu item
    @Override
    public boolean onActionItemClicked(ActionMode mode, MenuItem item) {
        switch (item.getItemId()) {
            case R.id.menu_share:
                shareCurrentItem();
                mode.finish(); // Action picked, so close the CAB
                return true;
            default:
                return false;
        }
    }
    // Called when the user exits the action mode
    @Override
    public void onDestroyActionMode(ActionMode mode) {
        mActionMode = null;
    }
};
```

Notice that these event callbacks are almost exactly the same as the callbacks for the options menu, except each of these also pass the ActionMode object associated with the event. You can use ActionMode APIs to make various changes to the CAB, such as revise the title and subtitle with setTitle() and setSubtitle() (useful to indicate how many items are selected).

Also notice that the above sample sets the mActionMode variable null when the action mode is destroyed. In the next step, you'll see how it's initialized and how saving the member variable in your activity or fragment can be useful.

2. Call startActionMode() to enable the contextual action mode when appropriate, such as in response to a long-click on a View:

```
someView.setOnLongClickListener(new View.OnLongClickListener() {

    // Called when the user long-clicks on someView

    public boolean onLongClick(View view) {

        if (mActionMode != null) {

            return false;

        }

        // Start the CAB using the ActionMode.Callback defined above

        mActionMode = getActivity().startActionMode(mActionModeCallback);

        view.setSelected(true);

        return true;

    }

});
```

When you call startActionMode(), the system returns the ActionMode created. By saving this in a member variable, you can make changes to the contextual action bar in response to other events. In the above sample, the ActionMode is used to ensure that the ActionMode instance is not recreated if it's already active, by checking whether the member is null before starting the action mode.

## Enabling batch contextual actions in a ListView or GridView

If you have a collection of items in a ListView or GridView (or another extension of AbsListView) and want to allow users to perform batch actions, you should:

- Implement the AbsListView.MultiChoiceModeListener interface and set it for the view group with setMultiChoiceModeListener(). In the listener's callback methods, you can specify the actions for the contextual action bar, respond to click events on action items, and handle other callbacks inherited from the ActionMode.Callback interface.
- Call setChoiceMode() with the CHOICE_MODE_MULTIPLE_MODAL argument.

For example:

```
ListView listView = getListView();
listView.setChoiceMode(ListView.CHOICE_MODE_MULTIPLE_MODAL);
listView.setMultiChoiceModeListener(new MultiChoiceModeListener() {
    @Override
    public void onItemCheckedStateChanged(ActionMode mode, int position,
                                          long id, boolean checked) {
        // Here you can do something when items are selected/de-selected,
        // such as update the title in the CAB
    }
    @Override
```

```
        public boolean onActionItemClicked(ActionMode mode, MenuItem item) {
            // Respond to clicks on the actions in the CAB
            switch (item.getItemId()) {
                case R.id.menu_delete:
                    deleteSelectedItems();
                    mode.finish(); // Action picked, so close the CAB
                    return true;
                default:
                    return false;
            }
        }
        @Override
        public boolean onCreateActionMode(ActionMode mode, Menu menu) {
            // Inflate the menu for the CAB
            MenuInflater inflater = mode.getMenuInflater();
            inflater.inflate(R.menu.context, menu);
            return true;
        }
        @Override
        public void onDestroyActionMode(ActionMode mode) {
            // Here you can make any necessary updates to the activity when
            // the CAB is removed. By default, selected items are deselected/unchecked.
        }
        @Override
        public boolean onPrepareActionMode(ActionMode mode, Menu menu) {
            // Here you can perform updates to the CAB due to
            // an invalidate() request
            return false;
        }
});
```

That's it. Now when the user selects an item with a long-click, the system calls the onCreateActionMode() method and displays the contextual action bar with the specified actions. While the contextual action bar is visible, users can select additional items.

In some cases in which the contextual actions provide common action items, you might want to add a checkbox or a similar UI element that allows users to select items, because they might not discover the long-click behavior. When a user selects the checkbox, you can invoke the contextual action mode by setting the respective list item to the checked state with setItemChecked().

# Creating a Popup Menu

A PopupMenu is a modal menu anchored to a View. It appears below the anchor view if there is room, or above the view otherwise. It's useful for:

- Providing an overflow-style menu for actions that *relate to* specific content (such as Gmail's email headers, shown in figure 4).

> **Note:** This is not the same as a context menu, which is generally for actions that *affect* selected content. For

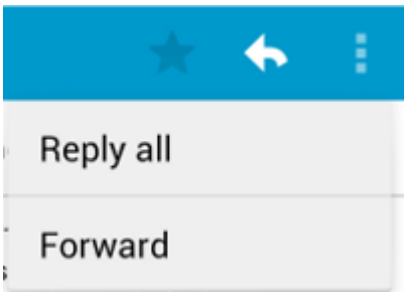



**Figure 4.** A popup menu in the Gmail app, anchored to the overflow button at the top-right.

> actions that affect selected content, use the contextual action mode or floating context menu.

- Providing a second part of a command sentence (such as a button marked "Add" that produces a popup menu with different "Add" options).
- Providing a drop-down similar to Spinner that does not retain a persistent selection.

> **Note:** PopupMenu is available with API level 11 and higher.

If you define your menu in XML, here's how you can show the popup menu:

1. Instantiate a PopupMenu with its constructor, which takes the current application Context and the View to which the menu should be anchored.
2. Use MenuInflater to inflate your menu resource into the Menu object returned by PopupMenu.getMenu().
3. Call PopupMenu.show().

For example, here's a button with the android:onClick attribute that shows a popup menu:

```
<ImageButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/ic_overflow_holo_dark"
    android:contentDescription="@string/descr_overflow_button"
    android:onClick="showPopup" />
```

The activity can then show the popup menu like this:

```
public void showPopup(View v) {
    PopupMenu popup = new PopupMenu(this, v);
    MenuInflater inflater = popup.getMenuInflater();
    inflater.inflate(R.menu.actions, popup.getMenu());
    popup.show();
}
```

In API level 14 and higher, you can combine the two lines that inflate the menu with PopupMenu.inflate().

The menu is dismissed when the user selects an item or touches outside the menu area. You can listen for the dismiss event using PopupMenu.OnDismissListener.

# Handling click events

To perform an action when the user selects a menu item, you must implement the PopupMenu.OnMenuItemClickListener interface and register it with your PopupMenu by calling setOnMenuItemclickListener(). When the user selects an item, the system calls the onMenuItemClick() callback in your interface.

For example:

```
public void showMenu(View v) {
    PopupMenu popup = new PopupMenu(this, v);
    // This activity implements OnMenuItemClickListener
    popup.setOnMenuItemClickListener(this);
    popup.inflate(R.menu.actions);
```

```
        popup.show();
    }
    @Override
    public boolean onMenuItemClick(MenuItem item) {
        switch (item.getItemId()) {
            case R.id.archive:
                archive(item);
                return true;
            case R.id.delete:
                delete(item);
                return true;
            default:
                return false;
        }
    }
```

# Creating Menu Groups

A menu group is a collection of menu items that share certain traits. With a group, you can:

- Show or hide all items with setGroupVisible()
- Enable or disable all items with setGroupEnabled()
- Specify whether all items are checkable with setGroupCheckable()

You can create a group by nesting <item> elements inside a <group> element in your menu resource or by specifying a group ID with the add()method.

Here's an example menu resource that includes a group:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/menu_save"
          android:icon="@drawable/menu_save"
          android:title="@string/menu_save" />
    <!-- menu group -->
    <group android:id="@+id/group_delete">
        <item android:id="@+id/menu_archive"
              android:title="@string/menu_archive" />
        <item android:id="@+id/menu_delete"
              android:title="@string/menu_delete" />
    </group>
</menu>
```

The items that are in the group appear at the same level as the first item—all three items in the menu are siblings. However, you can modify the traits of the two items in the group by referencing the group ID and using the methods listed above. The system will also never separate grouped items. For example, if you declare android:showAsAction="ifRoom" for each item, they will either both appear in the action bar or both appear in the action overflow.

# Using checkable menu items

A menu can be useful as an interface for turning options on and off, using a checkbox for stand-alone options, or radio buttons for groups of mutually exclusive options. Figure 5 shows a submenu with items that are checkable with radio buttons.

> **Note:** Menu items in the Icon Menu (from the options menu) cannot display a checkbox or radio button. If you choose to make items in the Icon Menu checkable, you must manually indicate the checked state by swapping the icon and/or text each time the state changes.

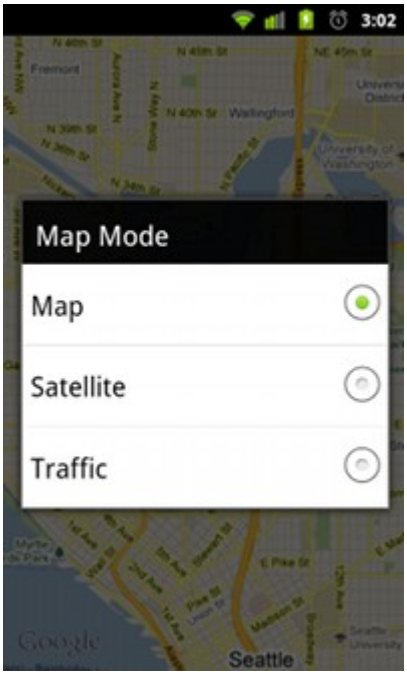



**Figure 5.** Screenshot of a submenu with checkable items.

You can define the checkable behavior for individual menu items using the android:checkable attribute in the <item> element, or for an entire group with the android:checkableBehavior attribute in the <group> element. For example, all items in this menu group are checkable with a radio button:

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <group android:checkableBehavior="single">
        <item android:id="@+id/red"
              android:title="@string/red" />
        <item android:id="@+id/blue"
              android:title="@string/blue" />
    </group>
</menu>
```

The android:checkableBehavior attribute accepts either:

single
: Only one item from the group can be checked (radio buttons)

all
: All items can be checked (checkboxes)

none
: No items are checkable

You can apply a default checked state to an item using the android:checked attribute in the <item> element and change it in code with the setChecked() method.

When a checkable item is selected, the system calls your respective item-selected callback method (such as onOptionsItemSelected()). It is here that you must set the state of the checkbox, because a checkbox or radio button does not change its state automatically. You can query the current state of the item (as it was before the user selected it) with isChecked() and then set the checked state with setChecked(). For example:

```
@Override
```

```
public boolean onOptionsItemSelected(MenuItem item) {
    switch (item.getItemId()) {
        case R.id.vibrate:
        case R.id.dont_vibrate:
            if (item.isChecked()) item.setChecked(false);
            else item.setChecked(true);
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
```

If you don't set the checked state this way, then the visible state of the item (the checkbox or radio button) will not change when the user selects it. When you do set the state, the activity preserves the checked state of the item so that when the user opens the menu later, the checked state that you set is visible.

> **Note:** Checkable menu items are intended to be used only on a per-session basis and not saved after the application is destroyed. If you have application settings that you would like to save for the user, you should store the data using Shared Preferences.

# Adding Menu Items Based on an Intent

Sometimes you'll want a menu item to launch an activity using an Intent (whether it's an activity in your application or another application). When you know the intent you want to use and have a specific menu item that should initiate the intent, you can execute the intent with startActivity() during the appropriate on-item-selected callback method (such as the onOptionsItemSelected() callback).

However, if you are not certain that the user's device contains an application that handles the intent, then adding a menu item that invokes it can result in a non-functioning menu item, because the intent might not resolve to an activity. To solve this, Android lets you dynamically add menu items to your menu when Android finds activities on the device that handle your intent.

To add menu items based on available activities that accept an intent:

1. Define an intent with the category CATEGORY_ALTERNATIVE and/or CATEGORY_SELECTED_ALTERNATIVE, plus any other requirements.
2. Call Menu.addIntentOptions(). Android then searches for any applications that can perform the intent and adds them to your menu.

If there are no applications installed that satisfy the intent, then no menu items are added.

> **Note:** CATEGORY_SELECTED_ALTERNATIVE is used to handle the currently selected element on the screen. So, it should only be used when creating a Menu in onCreateContextMenu().

For example:

```
@Override
public boolean onCreateOptionsMenu(Menu menu){
    super.onCreateOptionsMenu(menu);
```

```
    // Create an Intent that describes the requirements to fulfill, to be included
    // in our menu. The offering app must include a category value of Intent.CATEGORY_ALTERNATIVE.
    Intent intent = new Intent(null, dataUri);
    intent.addCategory(Intent.CATEGORY_ALTERNATIVE);
    // Search and populate the menu with acceptable offering applications.
    menu.addIntentOptions(
            R.id.intent_group,  // Menu group to which new items will be added
            0,      // Unique item ID (none)
            0,      // Order for the items (none)
            this.getComponentName(),   // The current activity name
            null,   // Specific items to place first (none)
            intent, // Intent created above that describes our requirements
            0,      // Additional flags to control items (none)
            null);  // Array of MenuItems that correlate to specific items (none)
    return true;
}
```

For each activity found that provides an intent filter matching the intent defined, a menu item is added, using the value in the intent filter's android:label as the menu item title and the application icon as the menu item icon. The addIntentOptions() method returns the number of menu items added.

> **Note:** When you call addIntentOptions(), it overrides any and all menu items by the menu group specified in the first argument.

# Allowing your activity to be added to other menus

You can also offer the services of your activity to other applications, so your application can be included in the menu of others (reverse the roles described above).

To be included in other application menus, you need to define an intent filter as usual, but be sure to include the CATEGORY_ALTERNATIVE and/or CATEGORY_SELECTED_ALTERNATIVE values for the intent filter category. For example:

```
<intent-filter label="@string/resize_image">
    ...
    <category android:name="android.intent.category.ALTERNATIVE" />
    <category android:name="android.intent.category.SELECTED_ALTERNATIVE" />
    ...
</intent-filter>
```

Read more about writing intent filters in the Intents and Intent Filters document.

For a sample application using this technique, see the Note Pad sample code.

### Load the XML Resource

When you compile your application, each XML layout file is compiled into a `View` resource. You should load the layout resource from your application code, in your `Activity.onCreate()` callback implementation. Do so by calling setContentView(), passing it the reference to your layout resource in the form of:R.layout.*layout_file_name*. For example, if your XML layout is saved as main_layout.xml, you would load it for your Activity like so:

```
// Code sample
public void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.main_layout);
}

// Code Sample ends
```

The onCreate() callback method in your Activity is called by the Android framework when your Activity is launched (see the discussion about lifecycles, in the Activities document).

## Android 3rd party UI/UX Libraries

library for UI/UX is a bundle tool of the components provided to ease out the development for specific application need. There are plenty of libraries have been created.

Few of them are as below

1) https://github.com/sd/AppMonk => Utility

2) https://github.com/cyrilmottier/GreenDroid => UI

3) https://github.com/zxing/zxing => (Barcode QR code)

4) https://code.google.com/p/jjil/ => Image processing

5) https://github.com/jgilfelt/android-mapviewballoons => UI (Google Map)